Tema 2.2:

Modelado de tópicos

Minería de textos, Grado en Inteligencia Artificial (GRIA), Universidad de Vigo
© Prof. David Candal Ventureira - 2026

Tabla de contenidos

1 Introducción

2 BERTopic

3 Evaluación

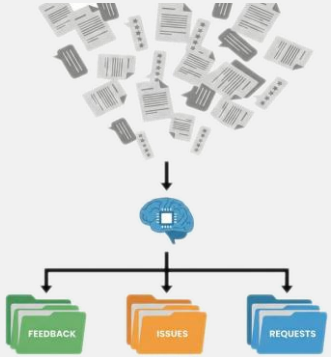
4 Adaptación

1

Introducción

Introducción: ¿Qué es el modelado de tópicos?

- Las empresas reciben miles o millones de textos que deben clasificar:
  - *Reviews* de un producto, *tickets* de soporte, análisis de tendencias en redes sociales…
- Etiquetar los documentos de manera manual es costoso y/o inviable.
- La empresa debe filtrar los textos a procesar por relevancia y priorizar las categorías que le interesen.



- **Modelado de tópicos**: Agrupa y etiqueta de manera no supervisada el corpus. Involucra:
  - ***Clustering***: Organización temática de los documentos en base a su estructura latente. Agrupa los documentos por su proximidad conceptual.
  - **Extracción de descriptores**: Determinar los términos más descriptivos de los documentos de cada clúster.

Introducción: Ejemplo de uso

- Escenario: Las valoraciones de nuestra app bajan drásticamente tras una actualización.
  1. Recogemos los nuevos comentarios negativos.
  2. Agrupamos los comentarios en base a su similitud (*clustering*).
     - Clúster 1: [“Feo”, “Malo”, “Inútil”, …].
     - Clúster 2: [“Cierra”, “Crash”, “Reinicia”, …].
  3. Extraemos los descriptores de cada clúster:
     - Clúster 1: Genérico.
     - Clúster 2: Inestabilidad.
  4. Resultado: Detectamos automáticamente que la nueva actualización de nuestra aplicación ha generado un problema de inestabilidad.

Introducción: Evolución histórica

1. **Latent dirichlet allocation** (LDA, 2000s): Basado en BoW y estadística (frecuencia de ocurrencia). Problema: Incapaz de agrupar correctamente por semántica, sólo agrupa por ocurrencias exactas.
2. **Enfoques basados en *embeddings*** (2019-2020): Capturan adecuadamente la semántica de los textos para la tarea de clasificación. Fallan a la hora de extraer los descriptores de los textos en los clústeres (encontrar palabras cercanas a los vectores textuales promedios de los clústeres).
3. **Enfoque modular** (2021-): Desacopla las fases de *clustering* y extracción de descriptores, permitiendo usar:
    - Algoritmos basados *embeddings* para *clustering*.
    - Algoritmos basados en BoW para extracción de descriptores.
    - (LLMs para un etiquetado final más descriptivo.)

Introducción: BERTopic

- Los sistemas de modelado de tópicos actuales, como **BERTopic**, son modulares.
- BERTopic consta de 5 fases:
  1. **Embeddings**: Convierte el texto en vectores (SBERT).
  2. **Reducción de dimensionalidad**: Comprime el espacio vectorial para facilitar el *clustering* (UMAP).
  3. ***Clustering***: Agrupa documentos similares y detecta ruido (HDBSCAN).
  4. **Tokenización**: Prepara el texto a nivel de *tokens* (CountVectorizer).
  5. **Representación**: Extrae los términos descriptores finales (c-TF-IDF).

- Ventajas del sistema modular:
  - **Flexibilidad**: Podemos actualizar cada módulo de manera independiente en función de los avances en el estado del arte.
  - **Escalabilidad**: Podemos destinar diferentes recursos a cada fase en producción.

2

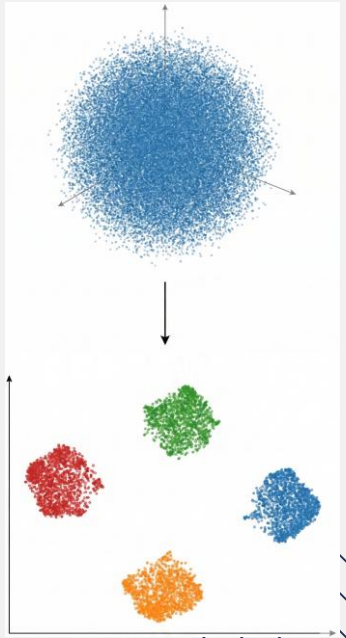
BERTopic

BERTopic (F1): Generación de *sentence embeddings*

- Procedimiento:
  1. Partimos de un corpus de documentos no etiquetado.
  2. Transformamos cada texto en *embeddings* con modelos como SBERT.
- Las representaciones resultantes tienen una alta dimensionalidad (ej: 768 dimensiones):
  - En **búsqueda semántica**, los espacios resultantes permiten obtener documentos muy relevantes a una consulta, devolviendo los documentos cercanos a un punto concreto.
  - En ***clustering***, los documentos están tan dispersos que no existen zonas vacías que aíslen los documentos. Recorriendo documentos vecinos, todo documento vecino a uno dado tiene nuevos vecinos no recorridos previamente, lo que hace muy difícil trazar fronteras que separen claramente los documentos por su semántica.

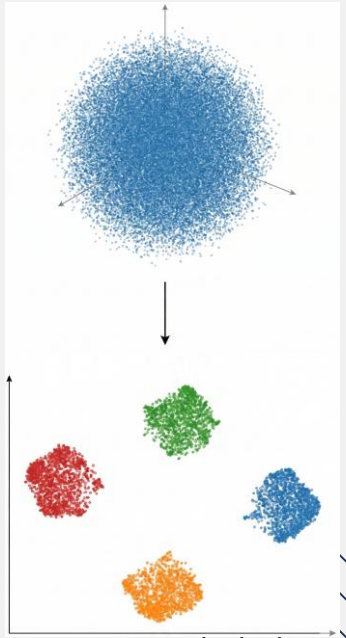
BERTopic (F2): Reducción de la dimensionalidad

- Objetivo: Proyectar el espacio de alta dimensionalidad a un **espacio de baja dimensionalidad** (ej: 5D o 10D), que cree **separaciones artificiales basadas en la estructura semántica**.
- Candidatos:
  - **PCA**: Proyección lineal del espacio -> No sólo no consigue aislar los clústeres, si no que mezcla los grupos que estaban separados en el espacio.
  - **t-SNE**:
    - La distancia entre clústeres no tiene significado -> No nos permite realizar fusión de tópicos o modelado de tópicos jerárquico.
    - La razón de similitud en base a la distancia entre elementos dentro de un clúster varía entre clústeres -> Conjuntos de documentos ruidosos pueden ser reconocidos como un clúster sólido.
    - El espacio resultante es impredecible. Requiere ejecutar todo el algoritmo para cada nuevo documento.



BERTopic (F2): Reducción de la dimensionalidad

- Objetivo: Proyectar el espacio de alta dimensionalidad a un **espacio de baja dimensionalidad** (ej: 5D o 10D), que cree **separaciones artificiales basadas en la estructura semántica**.
- Candidatos:
  - **UMAP**:
    - Tanto como la distancia entre clústeres como entre los documentos respetan la similitud semántica entre los pares -> Crea islas de documentos similares, claramente distanciadas.
    - El resultado de la ejecución es determinista y permite añadir nuevos documentos a un espacio ya generado.
    - Computacionalmente más ligero que t-SNE.



BERTopic (F2): Ajuste de hiperparámetros en UMAP

- UMAP requiere de una configuración previa, que debemos adaptar a nuestro problema (conseguir agrupar espacialmente los documentos en base a su similitud semántica):
  - **Número de vecinos (n_neighbors)**: Define el "rango de visualización del algoritmo". El algoritmo considera durante la reducción, para cada nodo, sus N vecinos más cercanos en el espacio original:
    - Valores bajos crean islas pequeñas -> Fragmentan tópicos.
    - Valores grandes crean "continentes" -> Riesgo de mezclar tópicos.
  - **Número de componentes (n_components)**: Dimensiones del espacio a generar. Habitualmente, 5/10 dimensiones para *clustering*.
  - **Métrica (metric)**: Usamos la similitud coseno, dado que es la más adecuada para determinar la similitud cuando trabajamos con lenguaje.
  - **Distancia mínima (min_dist)**: Fuerza a que, en el espacio resultante, cada nodo se encuentre a una distancia mínima de los demás (ej: para visualización). No nos interesa para *clustering* (usaremos valores muy bajos, como 0.0 - 0.1).

BERTopic (F2): Reducción de la dimensionalidad, UMAP

```
from bertopic import BERTopic
from umap import UMAP

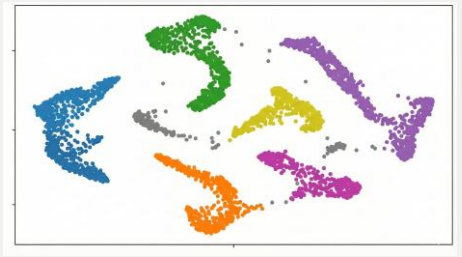
umap_model = UMAP(
    n_neighbors=15,        # Número de vecinos
    n_components=5,        # Número de componentes
    min_dist=0.0,          # Distancia mínima
    metric='cosine',       # Métrica
    random_state=0         # Semilla
)

# Usamos UMAP de manera aislada
corpus_reducido =
umap_model.fit_transform(embeddings)

print(f"Dim. originales: {embeddings.shape}")
print(f"Dim. reducidas: {corpus_reducido.shape}")
```

BERTopic (F3): *Clustering* de documentos textuales

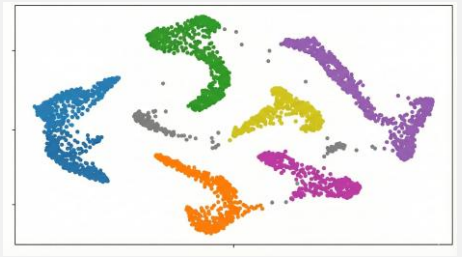
- Algoritmos simples como *k-means* definen centroides y agrupan los documentos en base a la distancia a estos puntos.
  - El lenguaje produce documentos donde los documentos pertenecientes a un mismo tema no se agrupan en torno a un punto del espacio, si no que los grupos producen **formas muy irregulares**.
  - *K-means* tiende a crear clústeres con tamaños similares. **La dispersión de los textos depende de su temática** (en ciertas temáticas, los textos son más cercanos entre si que en otras temáticas).
  - *K-means* fuerza a todos los documentos a ser clasificados en base al centroide más cercano -> **Fuerza a que el ruido contamine las categorías**.



- Solución: Agrupación en base a la **concentración de los puntos**. Identificar zonas de alta densidad rodeadas de un espacio de baja densidad (pocos documentos).

BERTopic (F3): *Clustering* de documentos textuales, HDBSCAN

- **HDBSCAN (*hierarchical density-based spatial clustering of applications with noise*)** es el algoritmo utilizado en industria para el *clustering* de documentos textuales y otro tipo de información ruidosa:
  - Agrupa los documentos en base a su **densidad**. Busca definir las **zonas donde existen multitud de documentos próximos**, definiendo el límite del clúster como el área donde la densidad de documentos decae rápidamente (islas vs océano).
  - **No asume una forma geométrica o densidad** a la hora de definir los clústeres.
  - Los **documentos ruidosos o ambiguos** (documentos sueltos en "el océano") **no se clasifican en base al clúster más cercano**, si no que se agrupan dentro de un clúster específico (clúster -1):
    - El algoritmo realiza una fase de **limpieza implícita**, definiendo clústeres limpios.
    - Los documentos ruidosos o ambiguos son identificados para un posible análisis manual (o simplemente para su descarte).



BERTopic (F3): Ajuste de hiperparámetros en HDBSCAN

- Como UMAP, HDBSCAN requiere de una configuración previa, que debemos adaptar a nuestro problema (conseguir agrupar espacialmente los documentos en base a su similitud semántica).
- El algoritmo agrupará los documentos en base a los siguientes hiperparámetros:
  - **Tamaño mínimo del tema (min_cluster_size)**: Número mínimo de elementos que pueden conformar un clúster:
    - Valores altos agrupan grandes corrientes temáticas como un único tema, pero pueden agregar temáticas que nos interesa mantener separadas.
    - Valores bajos pueden fragmentar demasiado el corpus.
  - **Mínimo de muestras (min_samples)**: Controla cuán conservador es el algoritmo al decidir qué zonas son lo suficientemente densas como para formar parte de un clúster y qué puntos son ruido:
    - Valores bajos generan más nodos nucleares dentro de cada clúster -> los clústeres introducen más documentos ambiguos/ruidosos.
    - Valores altos generan menos nodos nucleares dentro de cada clúster -> los clústeres atraen menos documentos ambiguos/ruidosos.

BERTopic (F3): Ajuste de hiperparámetros en HDBSCAN

- Como UMAP, HDBSCAN requiere de una configuración previa, que debemos adaptar a nuestro problema (conseguir agrupar espacialmente los documentos en base a su similitud semántica).
- El algoritmo agrupará los documentos en base a los siguientes hiperparámetros:
  - **Epsilon de selección (cluster_selection_epsilon)**: Distancia por debajo de la cual el algoritmo fusiona clústeres cercanos:
    - Valores altos hacen que clústeres más dispersos puedan fusionarse como un único clúster.
    - Valores bajos mantienen los clústeres iniciales aislados.

BERTopic (F3): *Clustering* de documentos textuales, HDBSCAN

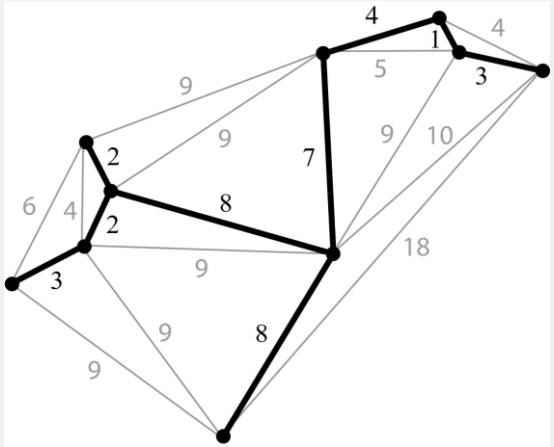
- Paso 1: Generación de un grafo basado en la **distancia de accesibilidad mutua**:
  1. El algoritmo comienza midiendo la distancia geométrica $d(A, B)$ entre cada par de documentos.
  2. Se calcula el **límite de vecindad (*core distance*, cd_A)** como la distancia al k vecino más cercano, con k = min_samples.
  3. Se calcula la **distancia de accesibilidad mutua (MRD)**, redefine la separación entre documentos, como:
     - $d_{mrd}(A, B) = \max(cd_A, cd_B, d(A, B))$
     - La MRD entre nodos ubicados en zonas densas será igual a su distancia geométrica. La MRD hacia y desde un nodo ubicado en una zona dispersa hacia un nodo cercano será mayor a su distancia geométrica, tomando el valor del mayor límite de vecindad de ambos nodos.
     - Los nodos más separados, aunque se encuentren cerca de una zona densa, quedan todavía más aislados al aumentar virtualmente su distancia al resto de nodos. Esto evita un **efecto cadena**: que nodos ruidosos no sólo se asignen a un clúster, si no que permitan a otros nodos (en cadena) ser asignados a dicho clúster.
  4. Este proceso genera un **grafo completo ponderado**, donde cada arista tiene como peso la distancia MDR.

BERTopic (F3): *Clustering* de documentos textuales, HDBSCAN

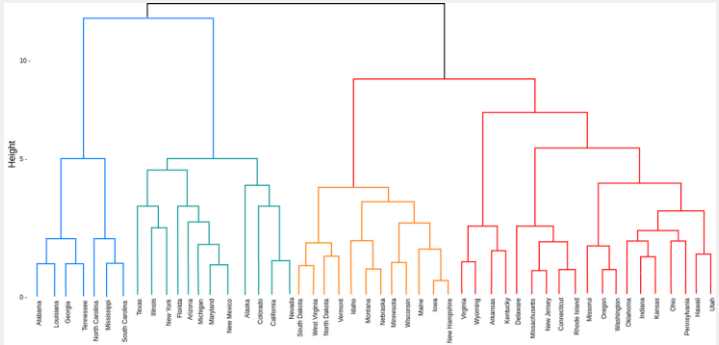
- Paso 2: **Construcción del árbol jerárquico**:
  - En base al grafo completo de distancias MRD, generamos el árbol **mínimum spanning tree (MST)**, que es el árbol sin ciclos que conecta todos nodos minimizando la suma total de las distancias.
    - En este árbol, las aristas con distancias pequeñas conectan nodos de un área temática, mientras que las aristas con distancias altas conectan con otros clústeres o con nodos ambiguos/ruidosos.
  - Transformamos el árbol MST en un **dendrograma**:
    - En la base (raíz) del dendrograma tenemos un único clúster que contiene todos los documentos.
    - Eliminamos iterativamente de nuestro árbol MST la **arista de mayor distancia**, rompiendo el árbol en dos clústeres cada vez más específicos.
    - Cada vez que dividimos el árbol MST, construimos una **bifurcación en el dendrograma**, donde cada nuevo nodo representa una de las nuevas componentes generadas en el árbol MST.
    - Continuamos hasta llegar a los nodos hoja del árbol MST (los documentos).
  - Problema: El dendrograma resultante es extremadamente detallado (demasiadas bifurcaciones). **Debemos condensarlo** para determinar los temas significativos.

BERTopic (F3): *Clustering* de documentos textuales, HDBSCAN

- Paso 2: **Construcción del árbol jerárquico**:



Ejemplo de árbol MST



Ejemplo de dendrograma

BERTopic (F3): *Clustering* de documentos textuales, HDBSCAN

- Paso 3: **Condensación y estabilidad**:
  - El dendrograma generado es demasiado complejo (tiene una bifurcación por cada división en el árbol MST).
  - Procedimiento de condensación:
    - Recorremos el dendograma desde la raíz hasta las hojas.
    - Para cada bifurcación, comprobamos el tamaño de los segmentos resultantes:
      - Si los dos segmentos tienen un tamaño superior a min_cluster_size, se anotan como potenciales clústeres (el padre se mantiene como candidato).
      - Si uno de los segmentos tiene un tamaño superior a min_cluster_size, pero el otro no, se descarta el segmento menor (se considera ruido), y se considera que **el segmento resultante es el mismo clúster que el original** (el padre), sin el ruido previamente existente.
    - Los clústeres candidatos (aquellos que no han sido excluidos al ser considerados ruido) se preservan para una selección posterior.

BERTopic (F3): *Clustering* de documentos textuales, HDBSCAN

- Paso 3: **Condensación y estabilidad**:
  - El dendrograma generado es demasiado complejo (tiene una bifurcación por cada división en el árbol MST).
  - Cálculo de la estabilidad de los clústeres:
    - En cada etapa, calculamos la estabilidad (S) de cada clúster candidato:
      - La **persistencia de nacimiento** (λ_{birth}) de un clúster es 1/d, donde d es la distancia de la arista que, al eliminarse, dio lugar al clúster.
      - Cada vez que se eliminan documentos del clúster (ruido), se calcula la **persistencia de muerte** (λ_{death}) de dichos documentos, como la inversa de la distancia de la arista que conllevó la bifurcación.
      - Si la bifurcación genera dos clústeres candidatos, se calcula la persistencia de muerte de todos los documentos restantes del clúster, a partir del inverso de la distancia de la arista que provocó la bifurcación.
      - La **estabilidad** (λ) de un clúster es la suma de las densidades de sus documentos, que es el resultado de la diferencia de la persistencia de muerte y la persistencia de nacimiento:
        - $S = \sum_{doc\ \epsilon\ clúster}(\lambda_{death} - \lambda_{birth})$

BERTopic (F3): *Clustering* de documentos textuales, HDBSCAN

- Paso 4: **Selección de clústeres**:
  - Cada clúster candidato (conjunto no descartado por ser considerado como ruido) tiene una **etiqueta de estabilidad**.
  - Procedimiento de selección de clústeres:
    - Partimos de las hojas y recorremos el dendograma hacia arriba.
    - En cada nivel jerárquico, el **conjunto de nodos hijos candidatos** a clúster **compite** con el **nodo de nivel superior**:
      - Si la **suma** de la estabilidad de los **hijos** es **mayor** que la estabilidad del nodo superior, **los nodos hijos se mantienen como candidatos** a clúster y se descarta la candidatura del nodo ascendente.
      - **Si la estabilidad del padre es superior**, se descarta la candidatura de todos los hijos y se mantiene la del padre.
    - Al subir al siguiente nivel en la jerarquía, se repite la operación con el siguiente nodo padre. Los nodos hijos con los que competirá serán aquellos que han ganado la contienda en su correspondiente bifuración (el padre o el hijo).
    - El algoritmo continua hasta alcanzar la raíz.
  - El resultado de esta operación es la **lista de clústeres** (pertenecientes a diferentes niveles en la jerarquía del dendrograma) que han superado el proceso de selección en base a la estabilidad contra sus ascendientes y descendientes.

BERTopic (F3): *Clustering* de documentos textuales, HDBSCAN

- En K-means, cada clúster se representa por su **centroide** (el promedio de los embeddings de cada clúster) -> Tenemos un vector numérico sin representación textual (no podemos pasarlo a palabras).
- En HDBSCAN, para cada clúster, el algoritmo determina un **medoide**:
  - El medoide de un clúster es el documento con el mayor valor de densidad dentro del clúster.
  - El medoide es considerado por el algoritmo el documento más central y representativo del clúster.
  - Un ingeniero puede analizar el medoide y sus documentos más cercanos para analizar la semántica del clúster.

BERTopic (F3): *Clustering* de documentos textuales, HDBSCAN

```python
from hdbscan import HDBSCAN
import numpy as np

hdbscan_model = HDBSCAN(
    min_cluster_size=15,            # Tamaño mínimo para considerar un clúster
    min_samples=2,                  # Permisividad con el ruido
    cluster_selection_epsilon=0.05, # Ajusta el grado en el que el algoritmo agrupa clústeres
    metric='euclidean',
    cluster_selection_method='eom', # 'eom' (Excess of Mass) es el estándar para tópicos
)

# Aplicamos HDBSCAN de manera aislada
clusters = hdbscan_model.fit_predict(corpus_reducido)
print(f"Número de tópicos encontrados: {len(np.unique(clusters))}")
print(f"Número de documentos marcados como ruido (clúster -1): {list(clusters).count(-1)}")
```

BERTopic (F4): Tokenización

- En las fases 1-3 hemos conseguido agrupar, de manera no supervisada, los documentos de entrada en base a su semántica (embeddings).
- En las fases 4-5 extraeremos los *tokens* de los documentos originales para cada clúster que mejor representen el contenido de cada uno de ellos (BoW).

- Tokenización:
  - **Concatenamos todos los documentos de cada clúster** en un único documento (documento de la clase).
  - Aplicamos la limpieza estándar: *stopwords*, n-gramas, lematización…
  - Convertimos cada clúster en un vector de frecuencias de *tokens*.

BERTopic (F5): Extracción de descriptores

- El TF-IDF tradicional busca términos que distinguen a un documento del resto de documentos en el corpus:
  - $W_{i,d} = tf_{i,d} \log(\frac{N}{df_i})$
- En modelado de tópicos necesita encontrar los términos que mejor identifican a todos los documentos de un clúster frente a los demás:
  - En lugar de tener miles de documentos, tenemos una decena.
  - Cada clúster tiene un tamaño (num. términos) diferente.
- **Class-based TF-IDF (c-TF-IDF)**:
  - $W_{i,c} = tf_{i,c} \log(1 + \frac{A}{f_i})$, donde:
    - $tf_{i,c}$ es la frecuencia de ocurrencia del término *i* en el clúster *c*.
    - f_i es la frecuencia de ocurrencia del término *i* en el corpus.
    - A es el número medio de términos que tiene un clúster.

BERTopic (F5): Extracción de descriptores, MMR

- Generalmente, c-TF-IDF nos devuelve las palabras más relevantes para representar un clúster, pero estas son **redundantes** (semánticamente muy relacionadas).
- Ej: ["coche", "vehículos", "automóvil", …] -> Aportan poca información.

- El algoritmo de re-ranking **MMR (*maximal marginal relevance*)** busca un equilibrio entre la **relevancia** de un término (c-TF-IDF) y su **diversidad** (diferencia semántica con los términos previamente seleccionados):
  - Se selecciona la palabra con mayor c-TF-IDF.
  - Iterativamente, seleccionamos el siguiente término más relevante.
  - El embedding de cada término extraído se compara con los embeddings de los términos aceptados. La relevancia del nuevo término se penaliza en base a la similitud con los embeddings de los términos ya aceptados.

$$MMR = \arg\max_{w \in C \setminus S} [(1 - \lambda) \cdot c\text{-}TF\text{-}IDF - \lambda \cdot \max_{s \in S}(sim_coseno(w, s))]$$

  - λ representa la diversidad (0 no penaliza similitud, 1 penalización severa).

- ["coche", "vehículos", "automóvil", …] -> ["coche", "carretera", "taller", …]

BERTopic: Ejemplo

```python
from bertopic import BERTopic
from umap import UMAP
from hdbscan import HDBSCAN
from sklearn.feature_extraction.text import
CountVectorizer
from bertopic.representation import
MaximalMarginalRelevance
import nltk
from nltk.corpus import stopwords

# Reducción de dimensionalidad (UMAP)
umap_model = UMAP(
    n_neighbors=15,
    n_components=5,
    min_dist=0.0,
    metric='cosine',
    random_state=42
)
# Clustering (HDBSCAN)
hdbscan_model = HDBSCAN(
    min_cluster_size=15,
    min_samples=2,
    cluster_selection_epsilon=0.05,
    metric='euclidean',
    cluster_selection_method='eom',
    prediction_data=True
)
```

```python
#  Tokenización (CountVectorizer) y diversificación de palabras (MMR)
nltk.download('stopwords', quiet=True)
stop_words_es = stopwords.words("spanish")
vectorizer_model = CountVectorizer(stop_words=stop_words_es)
representation_model = MaximalMarginalRelevance(diversity=0.3)

# BERTopic
topic_model = BERTopic(
    umap_model=umap_model,                # Estructura geométrica
    hdbscan_model=hdbscan_model,          # Definición de clústeres
    vectorizer_model=vectorizer_model,    # Limpieza de texto
    representation_model=representation_model, # Etiquetas diversas
    language="spanish",                   # Soporte multilingüe
    calculate_probabilities=True          # Para análisis detallado de ruido
)

# Resultado
topics, probs = topic_model.fit_transform(docs)
print(topic_model.get_topic_info())
```

3

Evaluación

Evaluación

Los sistemas de modelado de tópicos se evalúan a través de los siguientes mecanismos:

- **Coherencia de tópicos**: Mide la relación lógica entre los descriptores (palabras clave) de un tópico mediante el análisis de su contexto.
- **Análisis de *outliers***: Mide la calidad del *clustering* aplicado al corpus, analizando el ratio de documentos que han sido asignados al clúster -1 (ruido).
- **Métodos extrínsecos**: Analizan si el resultado es útil para el humano.

Evaluación: Coherencia de tópicos

Coherencia de tópicos: Mide la relación lógica entre los descriptores (palabras clave) de un tópico mediante el análisis de su contexto:

- Se recorre todo el corpus utilizando una ventana deslizante de longitud fija (ej: 100 tokens).
- Para cada ventana, se anotan los términos **del conjunto de términos relevantes de todos los clústeres**, obtenidos mediante c-TF-IDF, **que aparecen**.
- Se calcula la **información mutua puntual normalizada (NPMI)** de cada par de términos relevantes. Esta métrica determina el grado en el que dos palabras co-ocurren por una relación semántica real y no por puro azar.

Evaluación: Coherencia de tópicos

Coherencia de tópicos: Mide la relación lógica entre los descriptores (palabras clave) de un tópico mediante el análisis de su contexto:

- Cada término relevante construye un **vector de contexto**, que almacena los valores de NPMI del término frente al resto de términos relevantes de todos los clústeres.
- Se calcula la similitud coseno entre estos vectores de contexto. Esto permite validar que dos términos pertenecen al mismo tópico no sólo si aparecen juntos, sino si **aparecen en los mismos contextos**.
- La métrica de coherencia, C_v, es la media aritmética de los valores de similitud anteriores:
  - $C_v > 0.7$: Tópico excelente.
  - $0.3 < C_v < 0.7$: Tópico aceptable.
  - $C_v < 0.3$: Tópico incoherente o ruidoso.

Evaluación: Coherencia de tópicos

Coherencia de tópicos: Mide la relación lógica entre los descriptores (palabras clave) de un tópico mediante el análisis de su contexto:

- Al observar la co-ocurrencia de los términos relevantes en todo el corpus, buscamos asegurarnos que los términos tiendan a **aparecer juntos en el lenguaje general**, y que su relevancia no es simplemente un problema de ruido estadístico en uno de nuestros clústeres.
- NPMI cuantifica la **exclusividad** de un término. Penaliza los términos que aparecen en múltiples contextos (palabras comunes).
- Los vectores de contexto (vectores de NPMI) nos permiten analizar la similitud indirecta de dos términos, es decir, **si los términos tienden a aparecer junto a las mismas palabras** (mismo contexto).

Evaluación: Análisis de *outliers*

Análisis de *outliers*: Mide la calidad del *clustering* aplicado al corpus, analizando el ratio de documentos que han sido asignados al clúster -1 (ruido).

- Todo corpus textual es ruidoso (sarcasmo, erratas…).
- El sistema debe ser **capaz de discriminar documentos ruidosos o ambiguos** de documentos que pertenecen a una temática clara.
- Ratio de outliers:
  - **Bajo (<10%)**: Indica que el modelo es demasiado permisivo -> Los **clústeres son demasiado genéricos**, por lo que el modelo no puede determinar tópicos específicos (baja coherencia de tópicos).
  - **Alto (>50%)**: El modelo es demasiado estricto -> Puede dar lugar a **clústeres limpios y específicos** (alta coherencia de tópicos), pero será **incapaz de clasificar** un gran número de textos en producción.
- Si el sistema no discrimina correctamente los documentos ruidosos, debemos ajustar los hiperparámetros de UMAP (*n_neighbors* o *min_dist*) y HDBSCAN (*min_cluster_size* o *min_samples*).

Evaluación extrínseca

- Un modelo de tópicos es exitoso si genera resultados útiles para un humano:
  - Buscamos obtener términos que permitan a una persona identificar los documentos agrupados.
  - Los descriptores extraídos pueden no expresar bien a un humano la información que hay detrás del clúster de documentos.

- **Métodos extrínsecos**: Analizan si el resultado es útil para el humano:
  - **Validación por medoides**: Analizamos manualmente los documentos centrales de cada clúster para validar que los tópicos representan correctamente al clúster.
  - **Estabilidad de tópicos**: Aplicamos el modelo con diferentes subconjuntos de los datos y comprobamos si el resultado de cada ejecución es similar.
  - **Test de intrusión**: Se presenta a un humano la lista de términos relevantes para un tópico, añadiendo un término intruso. El humano debe detectar correctamente el término intruso.

4

Adaptación

Adaptación: Reducción de tópicos

- HDBSCAN define clústeres en base a densidad -> Puede aislar múltiples clústeres relacionados si existe suficiente separación entre ellos.
- Ej: Obtenemos múltiples clústeres como “Procesador”, “RAM”, “Disco duro”, cuando esperamos un único clúster “Componentes de PC”.

- **Fusión post-hoc basada en c-TF-IDF**:
  - Se calcula la similitud coseno entre los **vectores c-TF-IDF** (BoW) de cada clúster.
  - Se seleccionan los clústeres con **mayor similitud**.
  - Se **fusionan los clústeres** y se recalculan los vectores c-TF-IDF.
  - Se repite el proceso iterativamente.

- El procedimiento termina cuando:
  - Se alcanza un **número prefijado** de clústeres.
  - La **similitud máxima** entre clústeres no consigue superar un umbral predefinido.

Adaptación: Etiquetado generativo

- Tras aplicar las 5 fases de BERTopic, obtenemos una lista de **palabras inconexas**.
- Ejemplo: [freno, ruido, taller, pastillas].

- Podemos refinar el etiquetado de los clústeres utilizando una LLM que tome:
  - Los N términos más relevantes (c-TF-IDF + MMR).
  - Los M medoides de cada clúster.
  - Prompt (“En base a estos términos y estos documentos representativos, genera un título corto y profesional que represente este tópico”).

- Resultado:
  - [freno, ruido, taller, pastillas] -> "Tópico 14: Incidencias en sistemas de frenado"

Adaptación: Modelado de tópicos guiado

- Inyectamos conocimiento previo mediante **"palabras semilla"** para guiar la formación de clústeres específicos:
  - Definimos una lista de **términos representativos para cada tema** que deseamos encontrarnos.
    - Ej: “Atención al cliente” -> [“teléfono”, “espera”, “soporte”, “amable”]
    - “Aplicación” -> ["error", "cierra", "crash"]
  - Cada lista de términos genera un **vector de anclaje**, un embedding cercano a los documentos relevantes para el tema.
  - Los embeddings sintéticos que inyectamos ayudan que, durante la ejecución de UMAP, los documentos relevantes al tema se **mantengan juntos**.
  - Al generar un grafo donde los documentos se encuentran más juntos, HDBSCAN tiende a considerarlos como un **clúster único y representativo**.

Adaptación: Modelado de tópicos zero-shot

- Permite clasificar documentos en categorías predefinidas de forma **inmediata, sin ejemplos previos** para dichas categorías.
- En lugar de agrupar documentos por la densidad de puntos en un espacio, los agrupamos directamente por la **similitud con una lista de tópicos predefinida**:
  - Definimos una lista de tópicos.
    - Ej: [“Tecnología”, “Economía”, “Deportes”, …]
  - Para cada documento, calculamos su similitud con los vectores de los tópicos (su etiqueta).
  - Si un documento supera un umbral de similitud con una etiqueta, se le asigna automáticamente.

- **Enfoque híbrido**:
  - Utilizamos el enfoque zero-shot para documentos que encajan directamente con temáticas conocidas.
  - Procesamos los documentos que no encajan mediante BERTopic para descubrir nuevas temáticas.

¿Preguntas?